Python Code

```
import numpy as np
import matplotlib.pyplot as plt

def new_velocity(v, p_best, g_best, x, W=0.4, C1=2.05, C2=2.05):
    r1 = np.random.uniform(low=0.0, high=1.0, size=len(x))
    r2 = np.random.uniform(low=0.0, high=1.0, size=len(x))
    return (W * v) + (C1) * r1 * (p_best - x) +  (C2) * r2 * (g_best - x)

class Pso:

    def __init__(self, fitness_func, inf_limit, sup_limit, pop_size, genes, GERACOES=500, custo=True):
        self.pop  = np.random.uniform(low=inf_limit, high=sup_limit, size=(pop_size,genes))

        self.vel = np.random.uniform(low=-1.0, high=+1.0, size=(pop_size,genes))
        p_bests = np.array([x for x in self.pop])
        fitness_vals = np.array([fitness_func(x) for x in self.pop])
        g_best_idx = np.argmin(fitness_vals) #Esse é só o valor do idx do fitness!
        g_best = np.copy(p_bests[g_best_idx])
        fit_best = fitness_func(g_best)
        self.fitness_por_geracao = np.zeros(shape=GERACOES, dtype=float)
        self.geracoes = np.linspace(start=1, stop=GERACOES, num=GERACOES, dtype=int)

        for i in range(GERACOES): #Loop principal!
            print(i)
            for j in range(pop_size): #Loop da população!
                fit_atual = fitness_func(self.pop[j])
                if custo:
                    if fitness_vals[j] > fit_atual:
                        p_bests[j] = np.copy(self.pop[j]) # Salva uma cópia congelada!
                        fitness_vals[j] = fit_atual

                    if fit_best > fit_atual:
                        g_best = np.copy(self.pop[j]) # Salva uma cópia congelada!
                        fit_best = fit_atual
                else:
                    if fitness_vals[j] < fit_atual:
                        p_bests[j] = np.copy(self.pop[j]) # Salva uma cópia congelada!
                        fitness_vals[j] = fit_atual

                    if fit_best < fit_atual:
                        g_best = np.copy(self.pop[j]) # Salva uma cópia congelada!
```

```
                        fit_best = fit_atual

                self.vel[j] = new_velocity(self.vel[j], p_bests[j], g_best,
self.pop[j])
                self.pop[j] = self.pop[j] + self.vel[j]
            fitness_arr = np.array([fitness_func(ind) for ind in self.pop])
            melhor_idx = np.argmin(fitness_arr)
            self.fitness_por_geracao[i] = fitness_arr[melhor_idx]

    def plot(self):
        fig, ax = plt.subplots()

        ax.plot(self.geracoes, self.fitness_por_geracao)

        ax.set_xlabel("Geração")
        ax.set_ylabel("Fitness")
        ax.set_title("Algoritmo PSO")
        ax.legend(["Fitness"])

        plt.show()

```